CAPSTONE PROJECT 3 - 2026

DONE AND COMPILED BY OKILE M.OTIM

B-Safe -

Course-end Project 3

Description

Create a CI/CD Pipeline to convert the legacy development process to a DevOps process.

Background of the problem statement:
A leading US healthcare company, **Aetna**, with a large IT structure had a 12-week release cycle and their business was impacted due to the legacy process. To gain
true business value through faster feature releases, better service quality, and cost optimization, they wanted to adopt agility in their build and release process.
The objective is to implement iterative deployments, continuous innovation, and automated testing through the assistance of the strategy.

Implementation requirements:

1. Install and configure the Jenkins architecture on AWS instance
2. Use the required plugins to run the build creation on a containerized platform
3. Create and run the Docker image which will have the application artifacts
4. Execute the automated tests on the created build
5. Create your private repository and push the Docker image into the repository
6. Expose the application on the respective ports so that the user can access the deployed application
7. Remove container stack after completing the job

The following tools must be used:

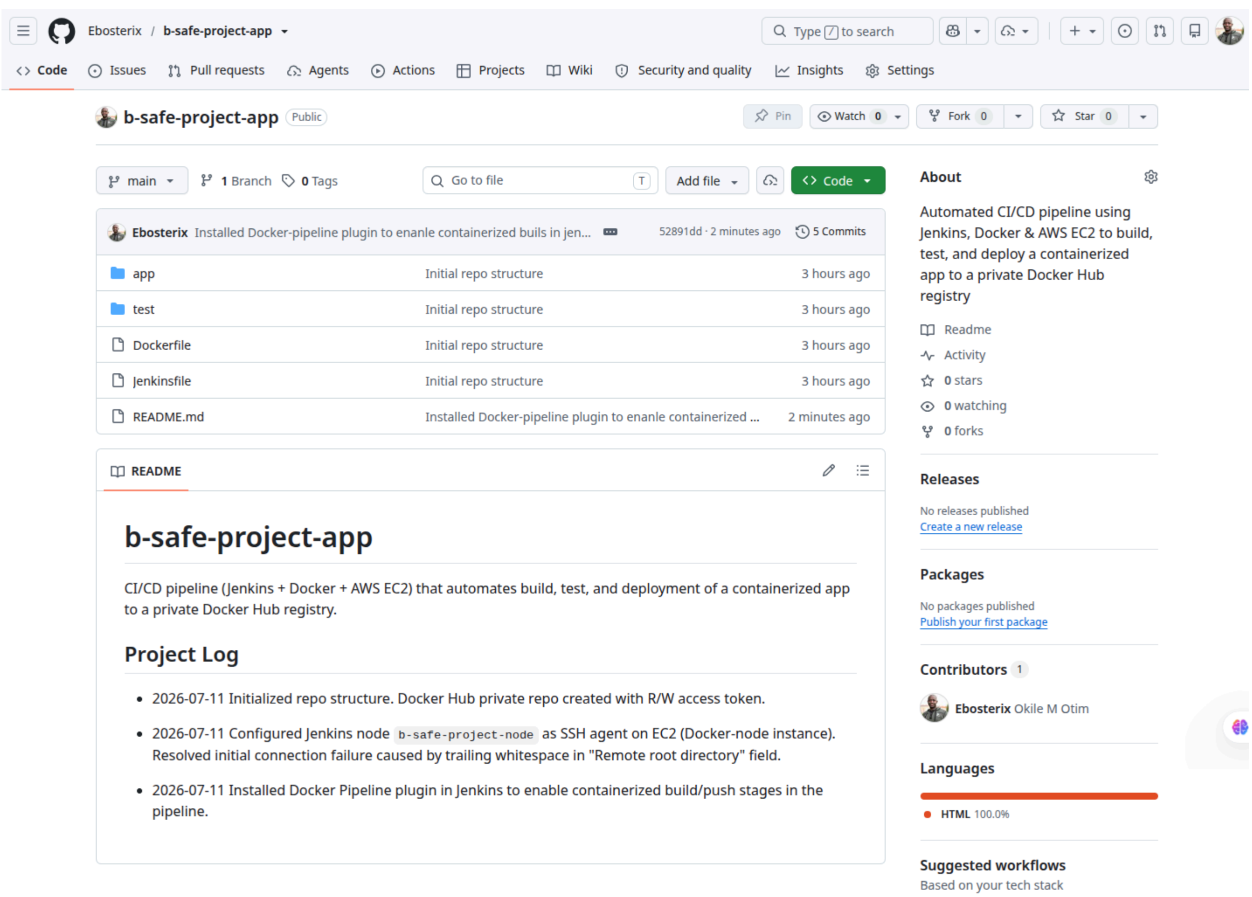
1. EC2
2. Jenkins
3. Docker
4. Git

The following things to be kept in check:

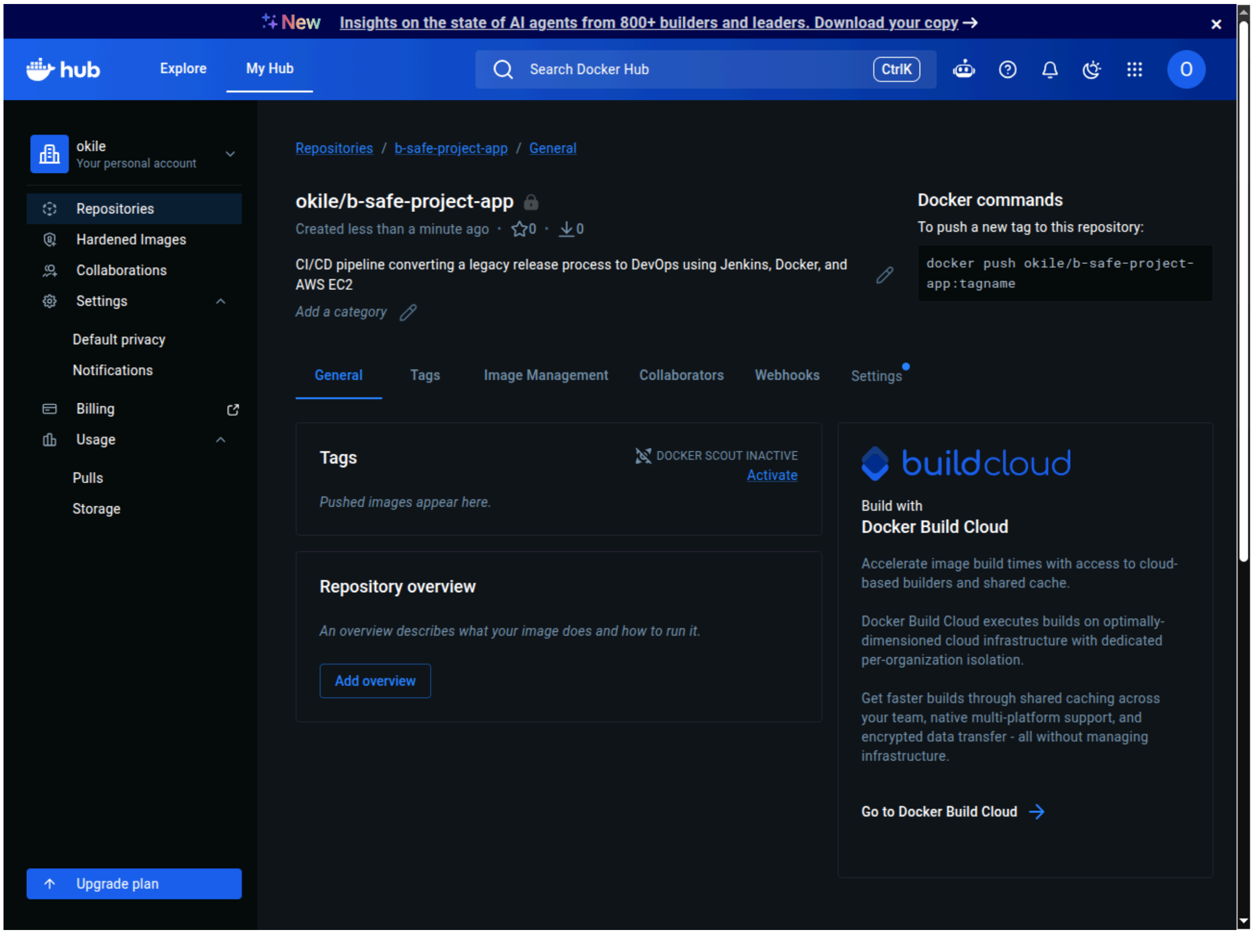
1. You need to document the steps and write the algorithms in them.
2. The submission of your Github repository link is mandatory. In order to track your tasks, you need to share the link of the repository.
3. Document the step-by-step process starting from creating test cases, the executing it, and recording the results.
4. You need to submit the final specification document, which includes:
    - Project and tester details
    - Concepts used in the project
    - Links to the GitHub repository to verify the project completion
    - Your conclusion on enhancing the application and defining the USPs (Unique Selling Points)

THE STEPS I TOOK TO DO THE ASSIGNMENT

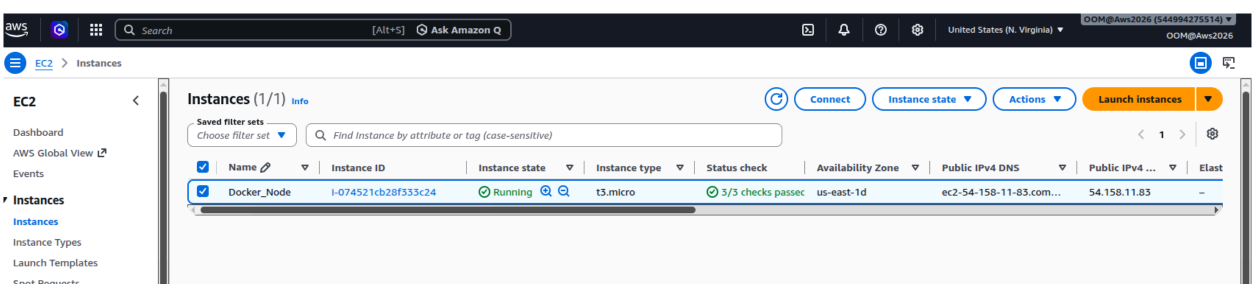
Step 2GitHub1: repo setup:

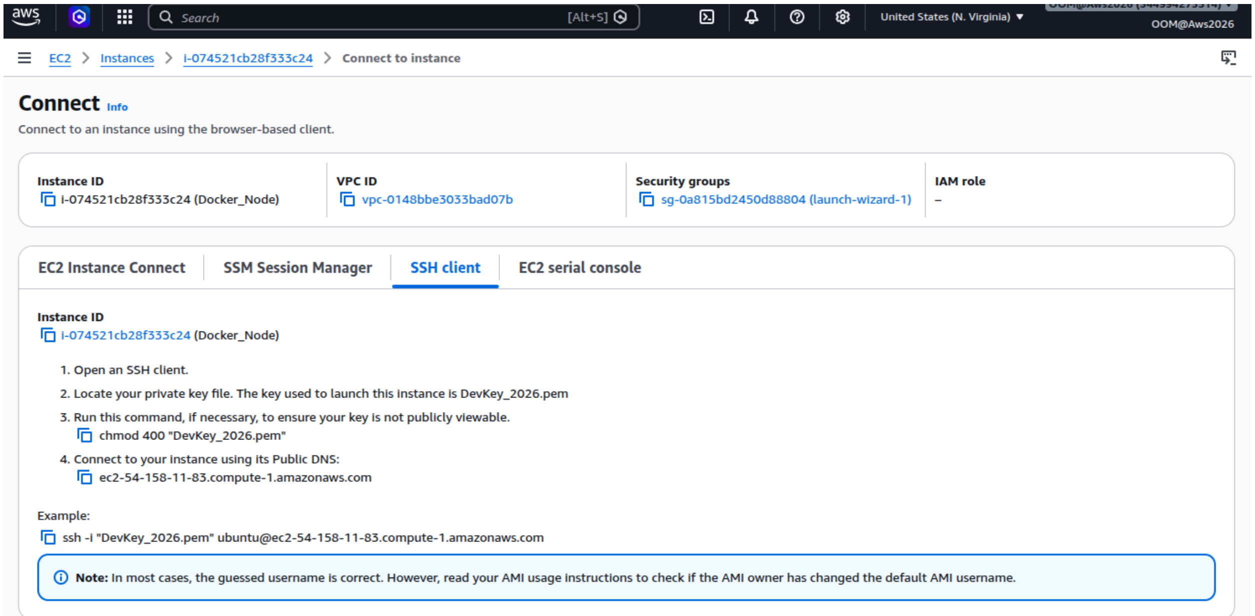


Docker Hub repo creation:

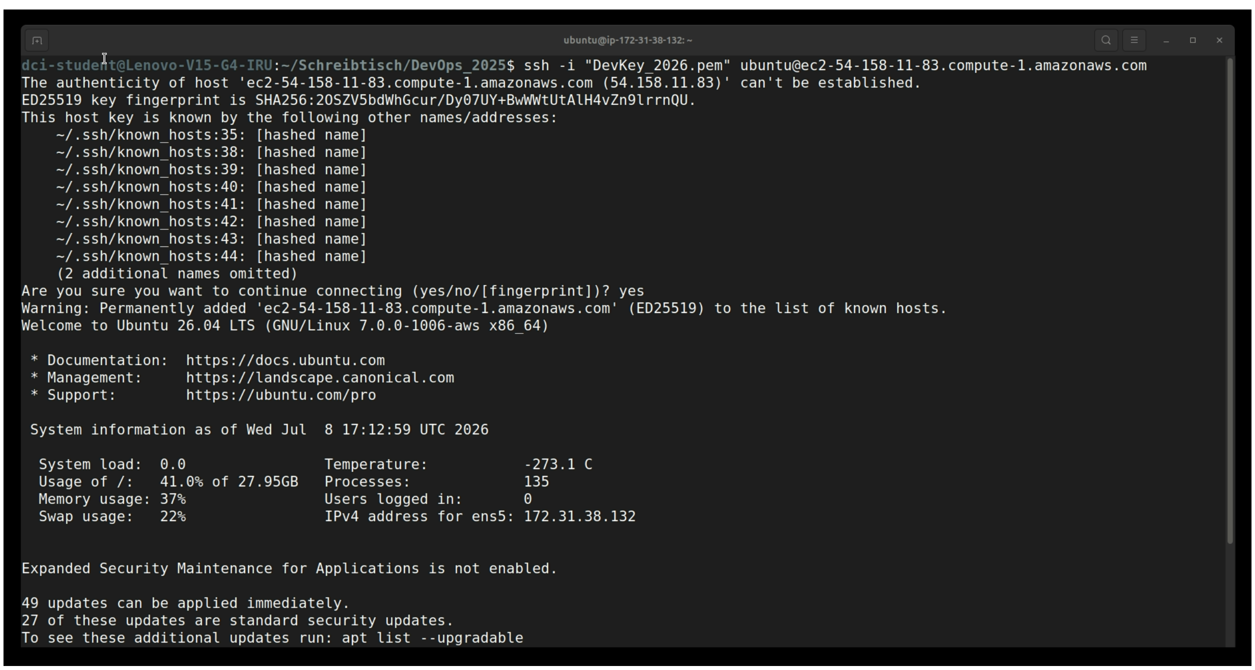


The EC2 instance





ssh -i "DevKey_2026.pem" ubuntu@ec2-54-158-11-83.compute-1.amazonaws.com



Logged into the Ubuntu Vm

```
Last login: Tue Jun 30 18:16:32 2026 from 91.67.84.180
ubuntu@ip-172-31-38-132:~$ java --version
openjdk 21.0.11 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.11+10-1-26.04.2-Ubuntu, mixed mode, sharing)
ubuntu@ip-172-31-38-132:~$
```

Step 2: Jenkins ↔ EC2 connection:

B-safe-project-node agent on Jenkins working

Jenkins / Nodes / b-safe-project-node / Log

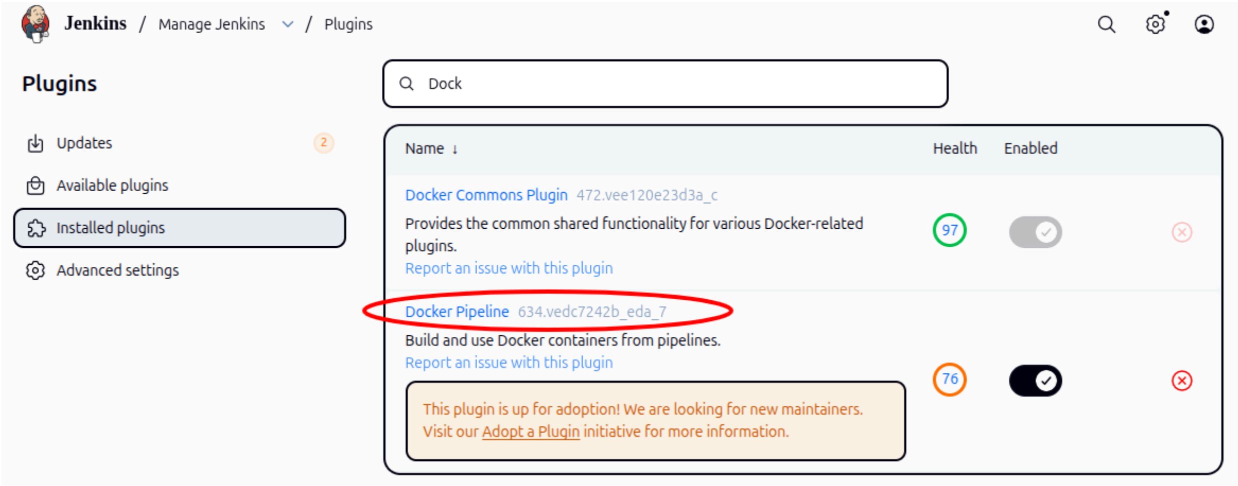
- Status
- Delete Agent
- Configure
- Build History
- Load Statistics
- Log

Build Executor Status 0/1

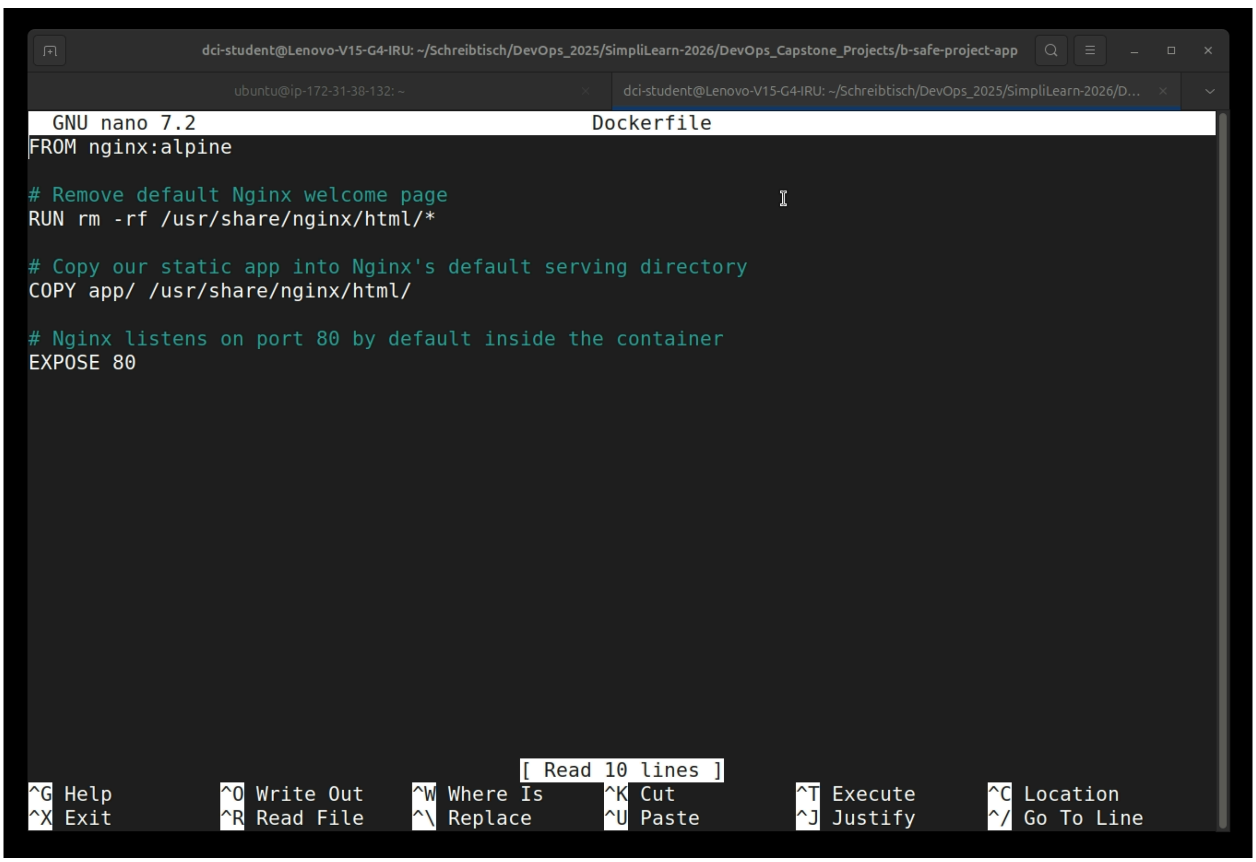
```
SSHLauncher{host='ec2-100-31-69-43.compute-1.amazonaws.com', port=22, credentialsId='b-
safe-ec2-ssh-key', jvmOptions='', javaPath='', prefixStartSlaveCmd='',
suffixStartSlaveCmd='', launchTimeoutSeconds=60, maxNumRetries=10, retryWaitTime=15,
sshHostKeyVerificationStrategy=hudson.plugins.sshslaves.verifiers.NonVerifyingKeyVerifica
tionStrategy, tcpNoDelay=true, trackCredentials=true}
[07/12/26 14:24:59] [SSH] Opening SSH connection to ec2-100-31-69-43.compute-
1.amazonaws.com:22.
[07/12/26 14:24:59] [SSH] WARNING: SSH Host Keys are not being verified. Man-in-the-
middle attacks may be possible against this connection.
[07/12/26 14:25:00] [SSH] Authentication successful.
[07/12/26 14:25:00] [SSH] The remote user's environment is:
BASH=/usr/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:extquote:force_fignore:globasciiranges:g
lobskipdots:hostcomplete:interactive_comments:patsub_replacement:progcomp:promptvars:sour
cepath
BASH_ALIASES=()
```

```
_=']'
[07/12/26 14:25:01] [SSH] Starting sftp client.
[07/12/26 14:25:02] [SSH] Copying latest remoting.jar...
Source agent hash is 7A9A77A64F02EB9F5DBCEFF9E9A349E2. Installed agent hash is
7A9A77A64F02EB9F5DBCEFF9E9A349E2
Verified agent jar. No update is necessary.
Expanded the channel window size to 4MB
[07/12/26 14:25:10] [SSH] Starting agent process: cd "/home/ubuntu/jenkins-agent" && java
-jar remoting.jar -workDir /home/ubuntu/jenkins-agent -jar-cache /home/ubuntu/jenkins-
agent/remoting/jarCache
Jul 12, 2026 12:25:11 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/ubuntu/jenkins-agent/remoting as a remoting work directory
Jul 12, 2026 12:25:11 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/ubuntu/jenkins-agent/remoting
channel started
Remoting version: 3355.v388858a_47b_33
Launcher: SSHLauncher
Communication Protocol: Standard in/out
Dies ist ein Unix- oder Linux-Agent
Agent successfully connected and online
```

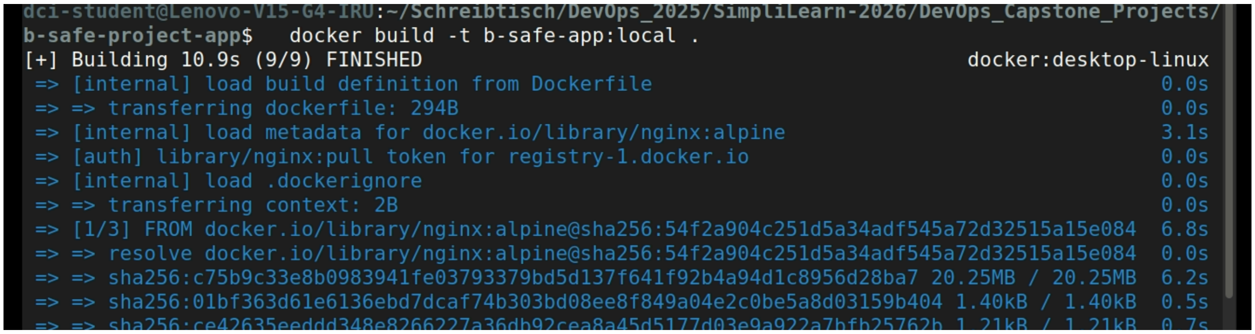
Step 3: Docker plugin installed

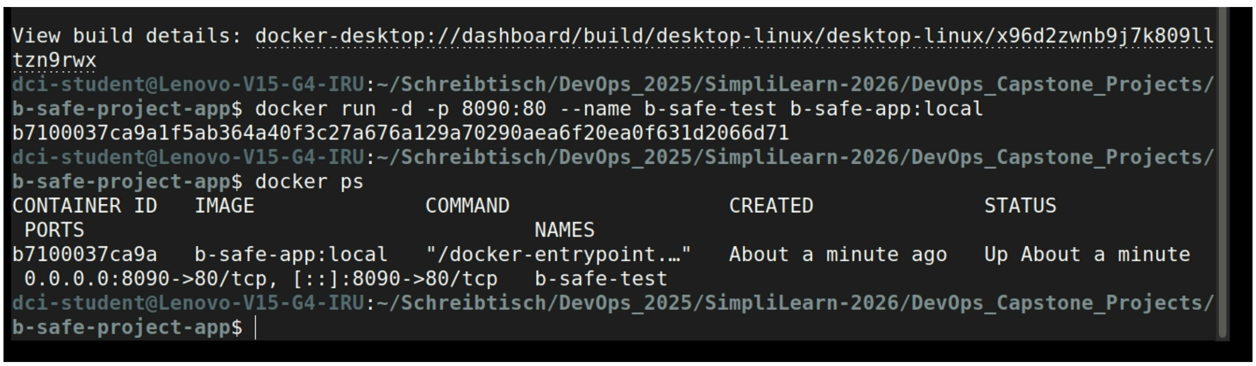


Step 4: Dockerfile created



Built and tested locally with Docker Desktop on port 8090: Successful:



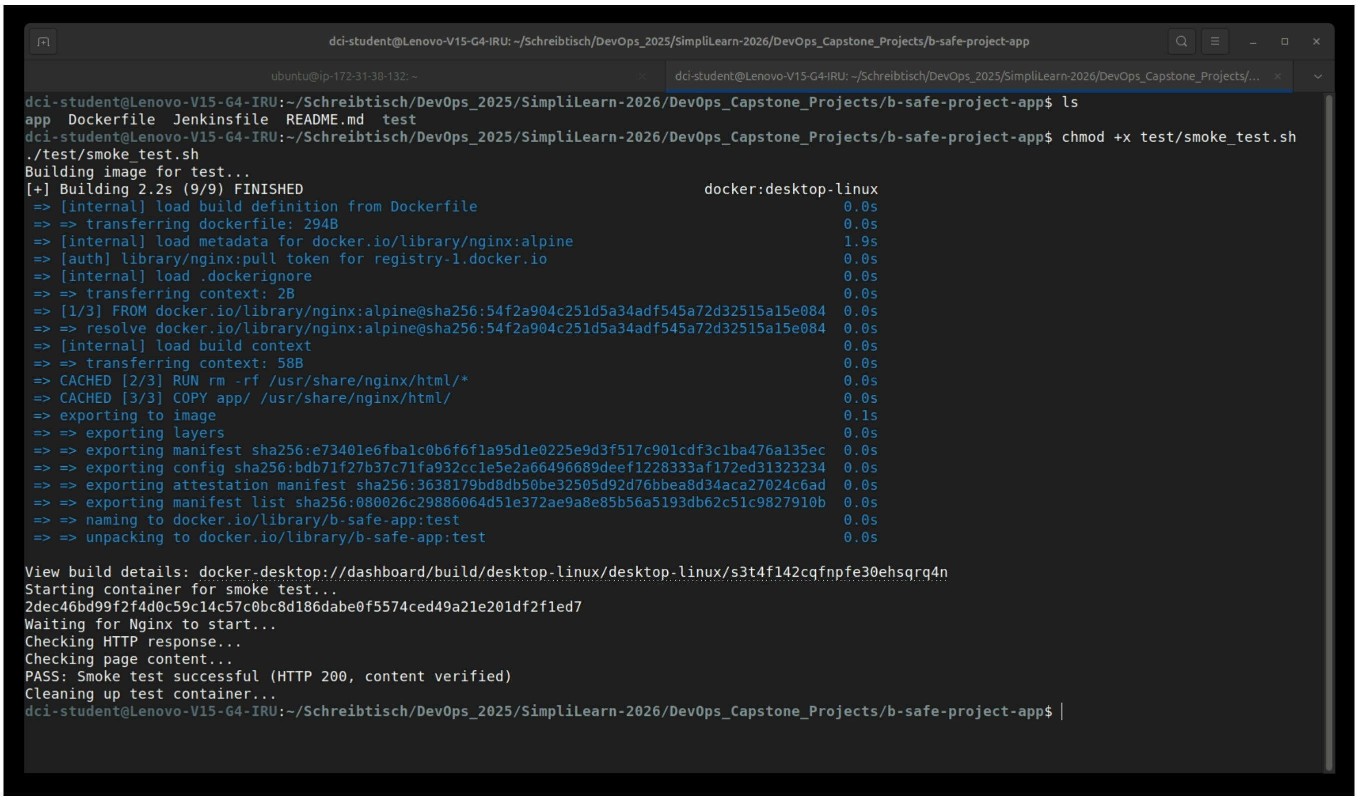


***** Clean up the local test container docker stop b-safe-test:**

docker stop b-safe-test

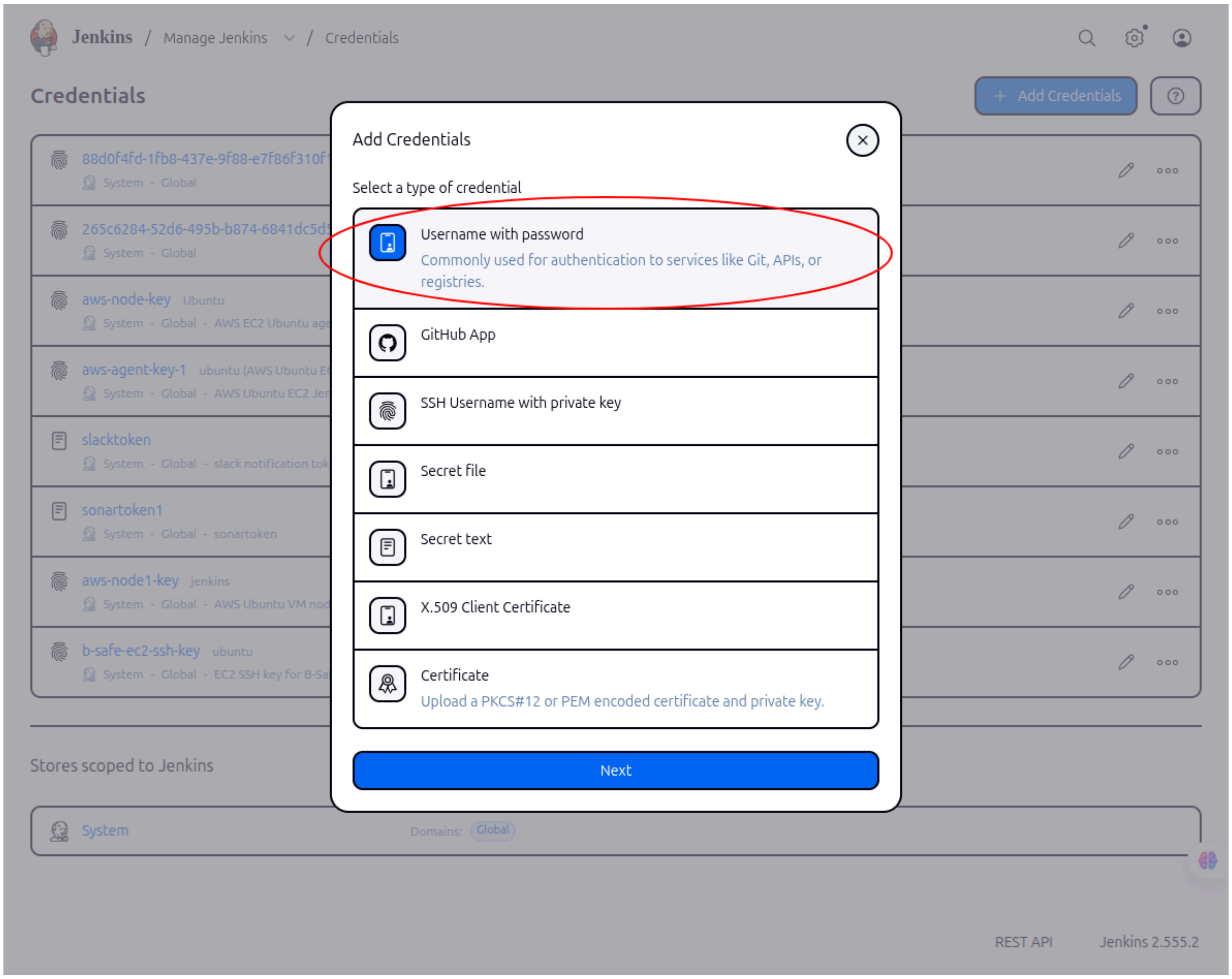
docker rm b-safe-test

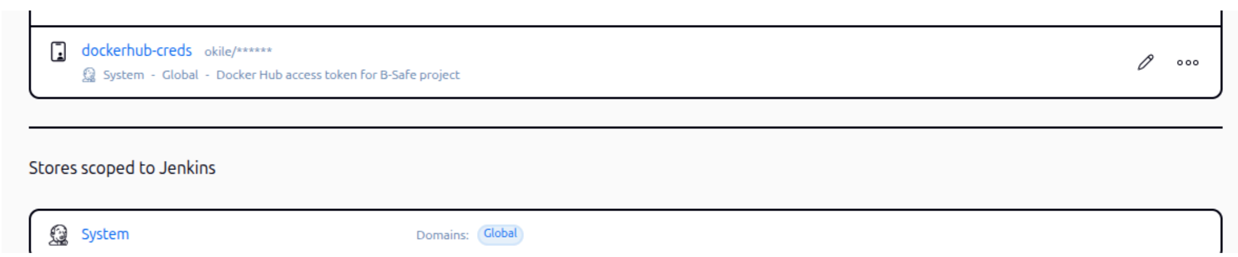
STEP 5: Automated test script: `smoke_test.sh` — curl/verify container

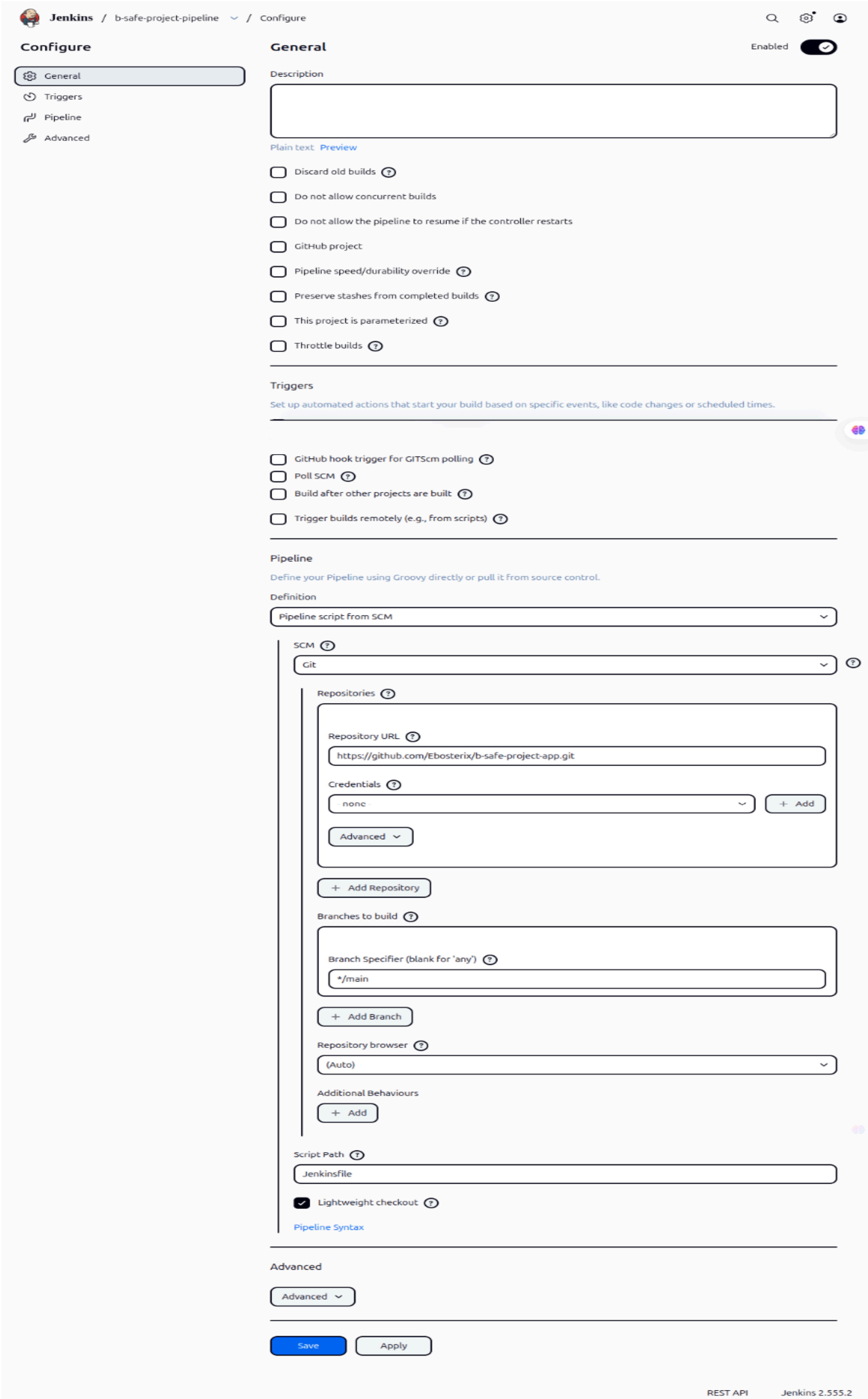


Step 6 — The Jenkinsfile

Set up `dockerhub-creds` in Jenkins

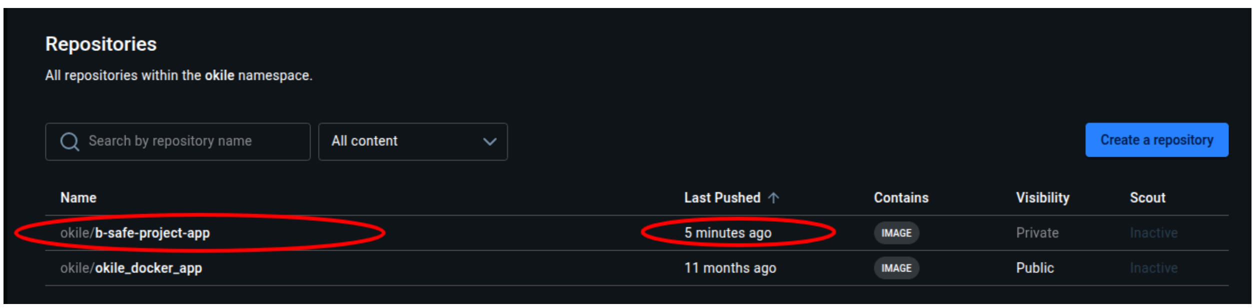

Jenkins / Manage Jenkins / Credentials
Credentials
Add Credentials
88d0f4fd-1fb8-437e-9f88-e7f86f310f
System - Global
265c6284-52d6-495b-b874-6841dc5d
System - Global
aws-node-key Ubuntu
System - Global - AWS EC2 Ubuntu ag
aws-agent-key-1 ubuntu (AWS Ubuntu E
System - Global - AWS Ubuntu EC2 Jen
slacktoken
System - Global - slack notification tok
sonartoken1
System - Global - sonartoken
aws-node1-key jenkins
System - Global - AWS Ubuntu VM nod
b-safe-ec2-ssh-key ubuntu
System - Global - EC2 SSH key for B-Sa
Add Credentials
Select a type of credential
Username with password
Commonly used for authentication to services like Git, APIs, or registries.
GitHub App
SSH Username with private key
Secret file
Secret text
X.509 Client Certificate
Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.
Next
Stores scoped to Jenkins
System
Domains: Global
REST API
Jenkins 2.555.2

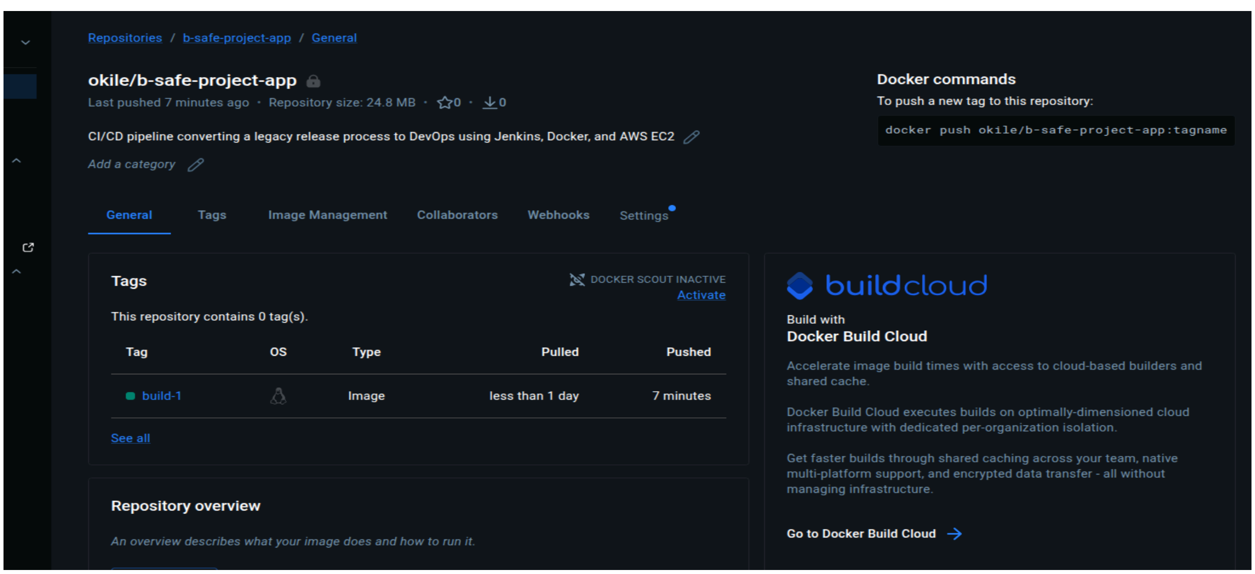

dockerhub-creds okile/******
System - Global - Docker Hub access token for B-Safe project
Stores scoped to Jenkins
System
Domains: Global


Jenkins / b-safe-project-pipeline / Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
Plain text Preview
Discard old builds
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Throttle builds
Triggers
Set up automated actions that start your build based on specific events, like code changes or scheduled times.
GitHub hook trigger for GITScm polling
Poll SCM
Build after other projects are built
Trigger builds remotely (e.g., from scripts)
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/Ebosterix/b-safe-project-app.git
Credentials
- none -
+ Add
Advanced
+ Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
+ Add Branch
Repository browser
(Auto)
Additional Behaviours
+ Add
Script Path
Jenkinsfile
Lightweight checkout
Pipeline Syntax
Advanced
Advanced
Save
Apply
REST API
Jenkins 2.555.2

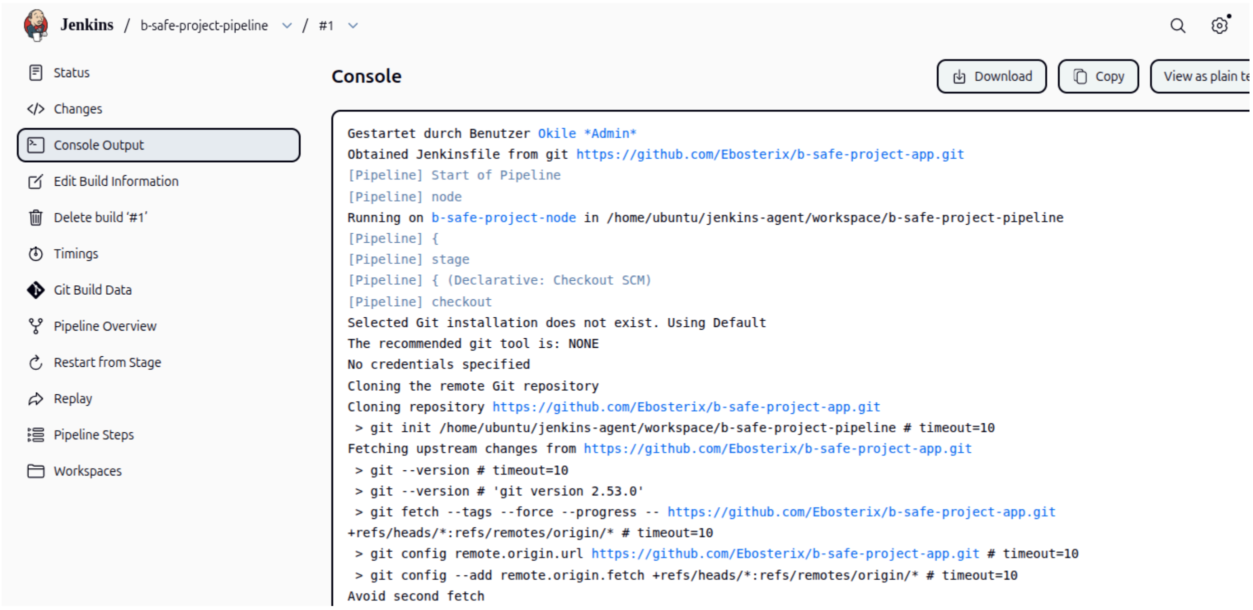
Jenkins job triggered successfully

Image pushed to docker hub





Step 7: First full pipeline run successfull

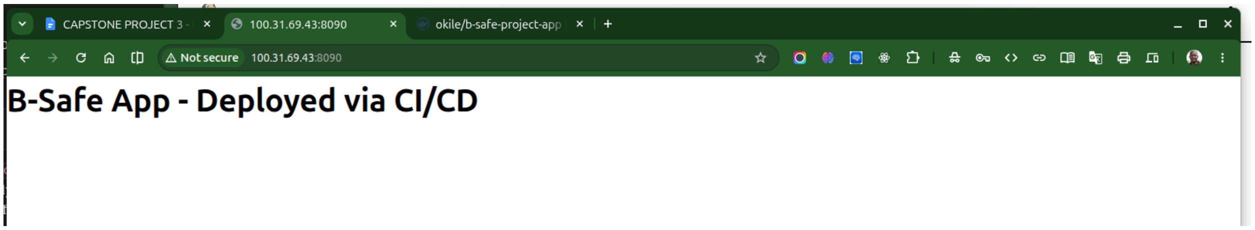



```
Gestartet durch Benutzer Okile *Admin*
Obtained Jenkinsfile from git https://github.com/Ebosterix/b-safe-project-app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on b-safe-project-node in /home/ubuntu/jenkins-agent/workspace/b-safe-project-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/Ebosterix/b-safe-project-app.git
 > git init /home/ubuntu/jenkins-agent/workspace/b-safe-project-pipeline # timeout=10
Fetching upstream changes from https://github.com/Ebosterix/b-safe-project-app.git
 > git --version # timeout=10
 > git --version # 'git version 2.53.0'
 > git fetch --tags --force --progress -- https://github.com/Ebosterix/b-safe-project-app.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/Ebosterix/b-safe-project-app.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
```

```
<h1>B-Safe App - Deployed via CI/CD</h1>
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Cleaning up dangling images...
[Pipeline] sh
+ docker image prune -f
Total reclaimed space: 0B
[Pipeline] echo
Pipeline completed successfully — app is live on port 8090.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Verify deployed app externally:

APP opens from EC2 domain: http://100.31.69.43:8090
This confirms the app is reachable from outside EC2 too, not just via localhost from inside the pipeline's own curl.



END